

## B 4.1.2 Evaluate linked lists (HL Only)

### B 4.1.2 Evaluate linked lists



Explore linked lists (singly, doubly, circular), their diagrammatic representation, and core operations: insertion, deletion, traversal, and search. We'll also evaluate their advantages and disadvantages, especially concerning memory use and performance compared to arrays.

### Introduction to Linked Lists

In computer science, **data structures** are fundamental ways of **organising and storing data** to efficiently perform operations on that data.

We have previously encountered **arrays** (and **lists** in Python), which are static or dynamic data structures that store elements in contiguous memory locations. This contiguous storage allows for efficient access to elements using an index. However, this fixed allocation or the need for shifting elements can make operations like insertion and deletion less efficient, particularly in the middle of a large array.

**Linked lists** offer an alternative approach to storing sequential data. Instead of storing elements contiguously, a linked list stores data in discrete units called nodes. Each **node** typically contains two parts: the data itself and a pointer (or reference) to the next node in the sequence. The list is accessed via a reference to the first node, often called the 'head'. This structure allows elements to be scattered throughout memory, with the pointers providing the logical links between them.

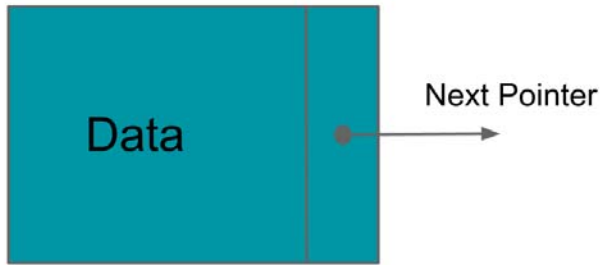
Evaluating linked lists involves understanding their structure, how basic operations are performed, and comparing their characteristics, particularly in terms of memory utilisation and performance, to other data structures like arrays.

### Types of Linked Lists

Linked lists come in several variations, distinguished by the direction of the pointers within the nodes. The primary types we will evaluate are singly, doubly, and circular linked lists.

#### Singly Linked Lists

The Node Structure (SLL)



A singly linked list is the most basic form. Each node contains the data and a single pointer, conventionally named 'next', which points to the subsequent node in the sequence.

The last node's 'next' pointer is typically `null` (or `None` in some programming languages) to signify the end of the list.

Access to the list begins at the 'head' node.

Traversal is possible only in one direction, from the head to the tail, by following the 'next' pointers sequentially.

## Doubly Linked Lists

Doubly Linked Lists (DLL) Node



A doubly linked list enhances the basic structure by adding a second pointer to each node. In addition to the 'next' pointer pointing to the successor node, each node also has a 'previous' pointer that points to the preceding node.

The 'previous' pointer of the head node and the 'next' pointer of the last node are typically `null`.

This structure allows for traversal in both directions – forwards from the head using 'next' pointers and backwards from the tail using 'previous' pointers.

The ability to traverse backwards simplifies certain operations, particularly deletion, as it provides direct access to the preceding node.

## Circular Linked Lists

A circular linked list is a variation where the last node's pointer does not point to `null`, but instead points back to the first node (the head).

This creates a circular structure where there is no explicit end to the list; traversal can continue indefinitely around the circle.

Circular linked lists can be either singly circular (each node has a 'next' pointer, and the last node's 'next' points to the head) or doubly circular (each node has 'next' and 'previous' pointers, with the head's 'previous' pointing to the last node and the last node's 'next' pointing to the head).

This structure is useful in scenarios where continuous looping through the data is required.

## Sketching Linked Lists and Basic Operations Diagrammatically

In diagrams, nodes are typically represented as boxes. Each box is divided to show the data part and the pointer(s) part.

Pointers are shown as arrows emanating from the pointer part of a node and pointing towards the node they reference. The 'head' reference is often shown as a separate arrow pointing to the first node.

Let's describe how basic operations would be depicted diagrammatically:

### Traversal

The goal of traversal is to visit each node in the list, typically starting from the head.

- Begin with a pointer variable, say `current`, referencing the head node.
- Process the data in the `current` node.
- Then, update `current` to point to the node referenced by `current`'s 'next' pointer (`current = current.next`).
- Repeat until `current` is `null` (for singly/doubly lists) or until you return to the starting node (for circular lists).

The presentation below shows how to represent that diagrammatically.

The goal of a search is to find a node containing a specific data value.

- Start at the head with a `current` pointer.
- Compare the data in the `current` node with the target value.
  - If they match, the node is found.
  - If not, move the `current` pointer to the next node.
- Repeat until the data is found or the end of the list is reached (indicated by `current` being `null` or returning to the start in a circular list without finding the value).

To draw the linked list show the `current` pointer moving through the nodes. Next to each node's data, indicate the comparison result (e.g., "Is data == target? No/Yes"). Stop the process when the target is found or the end is reached.

## Insertion

The goal of insertion is to add a new node into the list. This could be at the beginning, at the end, or in the middle.

*Inserting at the beginning:*

- Let the new node be called `newNode`.
- Set `newNode.next` to point to the current head.
- Update the head reference to point to `newNode`.
- For doubly linked lists, also set the old head's 'previous' pointer to `newNode` and `newNode.previous` to `null`.
- For circular lists, the last node's 'next' pointer also needs to be updated to point to `newNode`.

*Inserting in the middle:*

- Let the new node be called `newNode`.
- Find the node *before* the desired insertion point (let's call it `previousNode`).
- Set `newNode.next` to point to the node currently pointed to by `previousNode.next`.
- Then, set `previousNode.next` to point to `newNode`.
- For doubly linked lists, update pointers in four places: `newNode.next` points to `previousNode.next`; `newNode.previous` points to `previousNode`; the node that was originally after `previousNode` now has its `previous` pointer set to `newNode`; `previousNode.next` is set to `newNode`.

The presentation below shows how to represent that diagrammatically.

## Deletion

The goal of deletion is to remove a specific node from the list. This could be the head node, the last node, or a node in the middle.

- Find the node to be deleted (let's call it `nodeToDelete`) and the node *before* it (let's call it `previousNode`).
  - *Deleting the head:* Update the head reference to point to the second node (`head = head.next`). For doubly linked lists, set the new head's 'previous' pointer to `null`. For circular lists, update the last node's 'next' to point to the new head.
  - *Deleting in the middle or at the end:* Set `previousNode.next` to point to the node *after* `nodeToDelete` (`previousNode.next = nodeToDelete.next`). For doubly linked lists, if there is a node after `nodeToDelete`, update its 'previous' pointer to point to `previousNode`. Note that in a doubly linked list, you don't necessarily need `previousNode` if you have a reference to `nodeToDelete` itself, as you can access the previous node via `nodeToDelete.previous`.

The presentation below shows how to represent that diagrammatically.

## Advantages and Disadvantages of Linked Lists.

Understanding when to use a linked list versus an array requires evaluating their respective strengths and weaknesses, particularly concerning how they handle memory and how quickly basic operations can be performed.

### Memory Utilisation

**Linked Lists:** Nodes in a linked list are not necessarily stored in contiguous memory locations. Each node explicitly stores pointer(s) to link to other nodes. This means linked lists require extra memory storage for these pointers alongside the data. This can be a disadvantage in terms of memory overhead, especially if the data being stored is small compared to the size of the pointer.

**Arrays:** Arrays store elements in a single, contiguous block of memory. They do not require extra space per element for pointers, as the memory addresses of elements are implicitly linked by their position and the array's base address. This can be more memory efficient than linked lists, especially for large numbers of small data items. However, the *total* memory allocated for a static array is fixed at creation, even if it's not fully used. Dynamic arrays might allocate more space than immediately needed to accommodate future growth, potentially wasting memory temporarily.

### Performance

Performance is often evaluated in terms of time complexity using Big O notation, which describes how the execution time of an algorithm scales with the size of the input data.

### Accessing Elements

- **Arrays:** Accessing an element at a specific index in an array is very efficient, typically  $O(1)$ . This is because the memory address of any element can be directly calculated from its index and the array's base address (a property known as random access).
- **Linked Lists:** To access an element at a specific position (e.g., the  $n$ th element) in a linked list, you must start from the head and traverse the list node by node, following the pointers, until you reach the desired position. In the worst case (accessing the last element), this requires visiting every node. Therefore, accessing an element by position is  $O(n)$  for linked lists (sequential access). Searching for a specific *value* also typically requires traversal, making it  $O(n)$ .

### Insertion and Deletion

- **Arrays:** Inserting or deleting an element in the middle of an array can be costly. If you insert into or delete from the middle, all subsequent elements must be shifted to make space or fill the gap, maintaining contiguity. In the worst case (inserting/deleting at the beginning), this requires shifting nearly all elements, resulting in  $O(n)$  time complexity. Inserting into a static array that is full might require creating a new, larger array and copying all elements, also an  $O(n)$  operation.
- **Linked Lists:** Insertion and deletion in a linked list can be very efficient once the *location* for the operation is known. Adding a node at the beginning or deleting the head is  $O(1)$ . Inserting or deleting a node in the middle requires updating only a few pointers. If you have a reference to the node *before* the insertion/deletion point (and the node to delete for doubly lists), these operations are  $O(1)$ . However, finding the correct insertion/deletion point typically requires traversing the list, which is an  $O(n)$  operation. The total time complexity for insertion/deletion at an arbitrary position is thus usually dominated by the search/traversal time, making it  $O(n)$  overall. Doubly linked lists offer a slight advantage for deletion if you have a reference to the node to be deleted, as you can easily access the previous node using the 'previous' pointer ( $O(1)$  pointer updates after finding the node).

### Flexibility and Resizing

- **Arrays:** Static arrays have a fixed size, which can be restrictive. Dynamic arrays can grow, but resizing typically involves allocating a new, larger block of memory and copying all existing elements, which is a time-consuming  $O(n)$  operation.

- **Linked Lists:** Linked lists are inherently dynamic. Nodes can be easily added or removed as needed without requiring a contiguous memory block. The list can grow or shrink organically, making them very flexible when the amount of data is unknown or changes frequently. There is no need for costly resizing operations, as dynamic arrays might require.

## Summary

Here is a summary comparing arrays and linked lists in terms of memory and performance characteristics:

| Feature                  | Arrays (Dynamic)                                       | Linked Lists (Singly/Doubly)                      |
|--------------------------|--------------------------------------------------------|---------------------------------------------------|
| Memory Allocation        | Contiguous block                                       | Scattered nodes connected by pointers             |
| Memory Overhead          | Low (no pointers per element)                          | Higher (space needed for pointers in each node)   |
| Random Access (by index) | $O(1)$ - Very efficient                                | $O(n)$ - Requires traversal                       |
| Search (by value)        | $O(n)$ (linear scan)                                   | $O(n)$ (traversal)                                |
| Insertion/Deletion       | $O(n)$ (requires shifting elements)                    | $O(1)$ (if location known), $O(n)$ (if searching) |
| Resizing                 | Can be costly $O(n)$                                   | Dynamic, grows/shrinks easily, no costly resizing |
| Flexibility              | Limited by initial size, resizing is needed for growth | Very flexible in size                             |



